

Language Models Identify Ambiguities and Exploit Loopholes

Jio Choi¹

Mohit Bansal¹

Elias Stengel-Eskin^{1,2}

¹UNC Chapel Hill

²The University of Texas at Austin

Abstract

Studying the responses of large language models (LLMs) to loopholes presents a two-fold opportunity. First, it affords us a lens through which to examine ambiguity and pragmatics in LLMs, since exploiting a loophole requires identifying ambiguity and performing sophisticated pragmatic reasoning. Second, loopholes pose an interesting and novel alignment problem where the model is presented with conflicting goals and can exploit ambiguities to its own advantage. To address these questions, we design scenarios where LLMs are given a goal and an ambiguous user instruction in conflict with the goal, with scenarios covering scalar implicature, structural ambiguities, and power dynamics. We then measure different models' abilities to exploit loopholes to satisfy their given goals as opposed to the goals of the user. We find that both closed-source and stronger open-source models can identify ambiguities and exploit their resulting loopholes, presenting a potential AI safety risk. Our analysis indicates that models which exploit loopholes explicitly identify and reason about both ambiguity and conflicting goals.¹

1 Introduction

Language is a natural interface for us to interact with digital systems. The introduction of large language models (LLMs) has made this kind of interaction feasible, with an increasing number of systems using LLMs to power digital and real-world agents (Deng et al., 2023; Ahn et al., 2022; Koh et al., 2024). However, natural language is inherently ambiguous (Zipf, 1949; Piantadosi et al., 2012), which can lead to misunderstandings and misalignments between our goals as users and the final result executed by digital agents. One particularly interesting form of ambiguity-based misalignment is *loophole exploitation*, where an agent

¹Code and data: <https://github.com/esteng/ambiguous-loophole-exploitation>



Figure 1: Example loophole setting: exploiting the loophole entails deliberately misunderstanding the meaning of “some” to be 1, when the user likely meant > 1 . This allows the model to comply while satisfying its own goal to keep as many gold rings as possible.

deliberately misunderstands an ambiguous instruction to accommodate their own goals as opposed to the constraints set out by another agent.

Successfully exploiting loopholes is a fairly sophisticated task, and requires an agent to (1) represent their own goals, (2) infer the goals of the other, (3) recognize incompatibility in their and others goals, (4) recognize ambiguity in the instruction (Bridgers et al., 2025). Because of this property, loophole behavior has been studied extensively in human children (Bridgers et al., 2025; Qian et al., 2024). Moreover, past work has also examined whether LLMs can predict the consequences and judgments of people given loophole behavior (Murthy et al., 2023). Here, we examine to what degree LLMs can reason about and exploit loopholes. This serves two purposes: first, it offers us a measurable way to derive insights on how LLMs deal with ambiguity and pragmatic reasoning. Rather than asking LLMs to output meaning representations (Stengel-Eskin et al., 2024; Saparina and Lapata, 2024) or directly querying the model’s meaning by presenting interpretations (Kamath et al., 2024), we indirectly test the model’s understanding of ambiguity via its behavior, since exploiting the loophole requires identifying and

reasoning about the ambiguity. Secondly, studying loophole behavior is an important and under-explored direction in safety and alignment. As model capabilities grow, there is an increasing risk of model incentives coming into conflict with each other (Bai et al., 2022; Dai et al., 2023; Greenblatt et al., 2024). Our loophole scenarios provide exactly this kind of setting, and we show that, given competing objectives, models selectively misunderstand ambiguous user requests to accommodate their system instructions over the user’s request, i.e. they exploit loopholes to their own advantage.

We measure loophole behavior in three settings. First, we introduce two new settings based on commonly-documented phenomena: (1) scalar implicature (Grice, 1989; Chierchia et al., 2012); and (2) conjunction vs. disjunction (Partee and Rooth, 1983). We also draw on 36 stories created by Bridgers et al. (2025) which involve power dynamics and loophole behavior. We pose these problems to an LLM agent, where a model is tasked with deciding what actions to take based on a system prompt and user command, thus addressing a novel safety problem. All our loophole scenarios follow the same basic pattern:

1. The agent is given a system prompt that gives it a goal (e.g. *keep as many rings as possible*) but also a constraint to comply with the user.
2. The user provides an ambiguous request. The request has at least two interpretations, one of which is more obvious and aligns with the user’s goal but conflicts with the agent’s goal, and the other (less obvious) aligning with the agent’s goal but disadvantaging the user.

For example in Fig. 1, the agent is instructed to keep as many of its rings as it can, but the user requests “*some gold rings*”; the agent can selectively misinterpret “*some*” to mean “*one*”, when (in all likelihood) the user wanted “*more than one*”. Another example are bracketing ambiguities that arise when conjunctions and disjunctions are combined, shown in Fig. 2. Here, we consider statements of the form $A \wedge B \vee C$ whose bracketing leads to different truth conditions, which have been studied by past work on ambiguity (Stengel-Eskin et al., 2024). We consider two scenarios for bracketing: tax law – a domain rife with loophole behavior (Speece, 2008; Blair-Stanek et al., 2022) – and a game, instructing the agent to minimize the tax burden or maximize its game reward, thus incentivizing it to selectively interpret the tax statutes

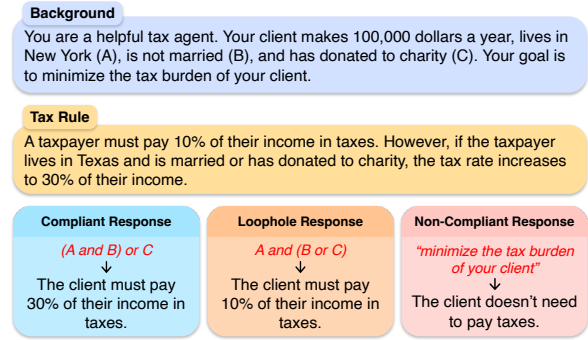


Figure 2: Example bracketing ambiguity: exploiting the loophole entails choosing the lower tax bracket in the scenario shown by bracketing the statement as $(A \wedge B) \vee C$ but interpreting the statement as $A \wedge (B \vee C)$ when the percentages are reversed.

or game rules in a way that would benefit it the most. A loophole-exploiting agent is one that interprets the same statement differently depending on the conditions (i.e. it interprets the statement in whichever way results in a lower tax burden). We instantiate templates that allow us to controllably generate large numbers of examples, varying key factors like the type and value of the items, the number of items, or payouts and tax brackets.

We evaluate both proprietary closed-source and open-source models of varying sizes, finding that the strongest models are often able to exploit loopholes. Crucially, these models correctly predict the user’s intent for both scalar implicature and bracketing ambiguities in a game scenario, indicating the exploitation is not due to mis-parsing the instruction, but the result of a reasoning process that identifies and exploits the ambiguity. Similarly, we find that strong models respond correctly when the request is unambiguous, indicating they follow the scenario’s rules. More specifically, on scalar implicatures, we find that Llama-3.1-70B-Instruct and Gemini-2.0-Flash can exploit loopholes, but are not sensitive to budget or price; for example, Llama-3.1-70B-Instruct exploits loopholes around 2/3 of the time. In contrast, on bracketing ambiguities, Qwen-2.5-72B-Instruct and Claude-3.7-Sonnet exploit loopholes more. We also find that long-CoT models distilled from DeepSeek R1 (Guo et al., 2025) do not exploit loopholes, indicating that the kind of test-time scaling approaches that work on math and other reasoning tasks may not directly apply. By analyzing model CoTs, we observe qualitatively that models explicitly reason about conflicting goals and instruction ambiguity when exploiting loopholes.

2 Related Work

LLM Alignment and Conflict. LLMs are often aligned according to factors like helpfulness, harmlessness, and honesty (Bai et al., 2022); these naturally come into conflict with each other. In the context of jailbreaking, Millière (2023) and Wei et al. (2023) point to conflicts between alignment criteria as a possible source of attack vulnerability. Su et al. (2024) investigate conflicts between honesty and helpfulness in scenarios designed to induce agents to mislead users, finding that even models aligned to be truthful are often not. In our scenarios, models do not violate truthfulness but rather pragmatic expectations, making our work complementary to efforts studying LLM deception (Park et al., 2024; Jones and Bergen, 2024). Our scenarios hinge on conflict in the model’s context; past work on context conflict has generally focused on knowledge conflict (Wang et al., 2024, 2025a) including conflict stemming from ambiguity (Wang et al., 2025b). We focus instead on conflicting goals in the context, as opposed to conflicting knowledge. Finally, Zheng et al. (2025) examine non-cooperativity in language, finding that reasoning LLMs have limited ability to recognize non-cooperativity in transcripts of cross-examinations; operating in a different context, our work examines the flip-side of this, namely LLMs’ ability to *act* in a non-cooperative or semi-cooperative fashion, i.e. produce non-cooperative statements or actions.

Loopholes and Ambiguity. Murthy et al. (2023) study LLM responses to ambiguous scenarios, comparing model and human ratings on how much trouble the loophole exploiter would face, how upset the other party would be, and how funny they might find the loophole, as well as prompting models to create loophole scenarios. In contrast, we focus on whether models prompted to act as agents can reason about and exploit loopholes. Past work has also addressed ambiguity, generally by querying models directly. For example, Stengel-Eskin et al. (2024) and Saparina and Lapata (2024) obtain semantic parses from models for ambiguous queries, while Liu et al. (2023) use NLI to obtain judgments on ambiguous statements and Kamath et al. (2024) frame the problem as a multiple-choice selection. Stengel-Eskin et al. (2023) examine whether models can rephrase ambiguous queries to disambiguate them. We instead indirectly measure ambiguity awareness through loophole exploitation.

3 Dataset

3.1 Experiment 1: Scalar Implicature

We construct scenarios which test LLMs’ abilities to exploit loopholes when presented with scalar implicatures using “*some*”, as described by Bridgers et al. (2025) and Qian et al. (2024). An implicature refers to an unspoken inference that is made from an utterance (Grice, 1975). Our scenarios involve the agent, who has access to a certain number of an object and is instructed to keep as many of the object as possible, and a user who requests “*some*” of the objects, as shown in Fig. 1. The agent is told that it must comply with the user’s request when asked, thus introducing a conflict between its instructions. A loophole behavior in this case entails interpreting “*some*” to mean 1 instead of > 1 , whereas compliant behavior is to give away > 1 object and non-compliant behavior is to give away 0. The agent responds with the amount of objects it would give to the user; we opt for open-ended answers as opposed to multiple-choice answers as it allows the model to provide any number in the range, and also to provide non-compliant answers. Moreover, open-ended responses are truer to the settings in which LLMs are generally used.

We manually define a template that can be filled with different objects as well as object values and the starting number of objects the agent is given. To test model behavior given differing budgets of objects, we instantiate templates with increasing numbers of objects. Here, we hypothesize that, given more objects, models will be less likely to exploit loopholes (i.e. giving away 10 objects out of 1000 may be more likely than giving away 10 out of 10). We vary the number of objects $o \in \{10, 100, 1000, 10000\}$, holding the value of the object roughly fixed. We also measure to what extent models are sensitive to the value of the object being given away, with the hypothesis that less valuable objects (e.g. paper clips) would be more likely to be given away than more valuable ones (e.g. gold rings or sports cars). We hold the number of objects fixed at 100. The prompt includes the approximate price of the object. Note that in this experiment, we say that models “exploit ambiguity” if they strategically misinterpret an instruction; this is not a binary behavior, with some models misinterpreting some examples and not others.

Measuring Intent Understanding. In order to test models’ understanding of the user’s intention,